TravelLink Backend — Progress Notes

Project Location

```
F:\Projects\TravelLink\Server\Server\
```

Tech Stack

- .NET 8, ASP.NET Core Web API
- Entity Framework Core + SQL Server (`TravelLinkDb`)
- JWT Authentication (`Microsoft.AspNetCore.Authentication.JwtBearer` v8.0.0)
- BCrypt.Net-Next (password hashing)
- CloudinaryDotNet (image storage)
- SignalR (real-time location)

Completed Modules

Entities (`Models/Entities/`)

- `User.cs` — includes `ImageUrl` , `CreatedGroups` nav property
- `FriendRequest.cs`
- `Group.cs` — includes `CoverImageUrl`
- `GroupMember.cs`
- `Expense.cs`
- `ExpenseSplit.cs`

AppDbContext (`Data/AppDbContext.cs`)

- All 6 DbSets registered
- Full Fluent API configured (dual FK Restrict on FriendRequest, Cascade on Group/Expense/ExpenseSplit)

Database

- `appsettings.json` — connection string + Jwt section + Cloudinary section all correct
- `InitialCreate` migration applied — `TravelLinkDb` live with all 6 tables

Common (`Common/ServiceResult.cs`)

- Generic `ServiceResult<T>` with `Success` , `Message` , `Data`

- Static factory methods: `Ok(data)` and `Fail(message)`
- Used by all Expense service methods for structured error messages → toaster UI

Program.cs

- DbContext, JWT, CORS, Cloudinary Singleton, SignalR all registered
- All services: `IAuthService`, `IFriendService`, `IGroupService`, `IImageService`, `IExpenseService`
- `UseAuthentication()` + `UseAuthorization()` in correct order
- `app.UseCors("auth")` with `WithOrigins("http://localhost:4200")` + `AllowCredentials()` for SignalR
- `app.MapHub<LocationHub>("/hubs/location")` registered
- JWT `Events.OnMessageReceived` reads token from `?access_token=` query string for SignalR WebSocket

Auth Module

- Register / Login endpoints tested & working

Friend System Module

Endpoint	Route
Send request	POST `/api/friend/send`
Accept/Reject	POST `/api/friend/respond/{id}?action=accept|reject`
Pending requests	GET `/api/friend/pending`
Friends list	GET `/api/friend/list`
Unfriend	DELETE `/api/friend/unfriend/{targetUserId}`

Design decisions:

- Unfriend blocked if unsettled **1-on-1** expenses exist between users
- Group expenses do NOT block unfriending

Group Module

- Auto-deletes group when last member leaves
- Admin transfer when admin leaves and no other admin exists | Endpoint | Route | |---|---| | Create group | POST `/api/group/create` | | My groups | GET `/api/group/my` | | Get group by ID | GET `/api/group/{id}` | | Add member (Admin) | POST `/api/group/{id}/add-member` | | Leave group | DELETE `/api/group/{id}/leave` | | Remove member (Admin) | DELETE `/api/group/{id}/remove-member` | | Upload group cover (Admin) | POST `/api/group/{id:guid}/cover` |

Image Module (Cloudinary)

- `IImageService` / `ImageService` (UploadImageAsync, DeleteImageAsync)

User Module

Endpoint	Route
Get profile	GET `/api/user/profile`
Upload/Update avatar	POST `/api/user/avatar`

Expense Module — COMPLETE

Design Decisions

- `SplitType: "Equal"` or `"Custom"` — validated early, rejects anything else
- `ParticipantIds` for 1-on-1 equal splits (no GroupId)
- Payer NOT force-added to splits — supports "paid for others" scenario
- Warning field in `ExpenseDto` when payer not in their own split
- Rounding remainder assigned to last participant
- Payer's own split auto-marked `IsPaid = true`
- Delete blocked if any participant already settled
- Balance netting: eliminates mutual debts
- **Friends-only rule**: 1-on-1 expenses only with accepted friends (group exempt)

DTOs (`DTOs/Expenses/`)

- `CreateExpenseDto.cs` , `ExpenseSplitInputDto.cs` , `ExpenseSplitDto.cs`
- `ExpenseDto.cs` (with `Warning?`), `BalanceDto.cs`
- `GroupAnalyticsDto.cs` , `UserToUserDto.cs`

Services

- `IExpenseService.cs` + `ExpenseService.cs` — all 8 methods return `ServiceResult<T>`

Controller

Endpoint	Route
Add expense	POST `/api/expense/add`
Get by ID	GET `/api/expense/{id}`
Delete	DELETE `/api/expense/{id}`
Group expenses	GET `/api/expense/group/{groupId}`
Group balances	GET `/api/expense/group/{groupId}/balances`
Group analytics	GET `/api/expense/group/{groupId}/analytics`
Settle split	PUT `/api/expense/split/{splitId}/pay`
User-to-user summary	GET `/api/expense/user/{userId}/summary`

Location Sharing Module — COMPLETE (Backend only)

Design Decisions

- Pure relay — NO DB writes per location update (stateless hub)
- Max 5 devices per session
- One active session per user at a time (auto-leaves previous on new join)
- Group session: membership checked against `GroupMembers` table
- 1-on-1 session: friendship checked against `FriendRequests` table
- Session names: `location-group-{groupId}` and `location-private-{sortedId1}-{sortedId2}`
- No DB migration needed

Files Created

- `DTOs/Location/LocationUpdateDto.cs`
- `Hubs/LocationHub.cs`

Hub Methods

Method	What it does
`JoinGroupSession(groupId)`	Verify member → auto-leave old → join group room
`JoinPrivateSession(targetUserId)`	Verify friends → auto-leave old → join private room
`SendLocation(dto)`	Broadcast to all others in session
`LeaveSession()`	Voluntarily exit, notify others
`OnDisconnectedAsync`	Auto-cleanup on crash/browser close

Client Events (Angular listens for these)

Event	When
`LocationUpdate`	Another user sent coordinates
`UserJoined`	Someone joined
`UserLeft`	Someone disconnected
`SessionLeft`	You left confirmation
`Error`	Auth failure or invalid action

Frontend Strategy (Angular — not built yet)

- `watchPosition()` with `enableHighAccuracy: true`
- Send only if moved > 10m AND 3s since last send
- Stationary > 60s → heartbeat every 30s
- JWT passed as `?access_token=` in hub connection URL

➡ Remaining Modules

1. AI Itinerary Builder (Gemini API) ← NEXT

- Gemini API integration
- Prompt engineering for day-by-day trip itinerary
- Endpoint: `POST /api/itinerary/generate`
- Input: destination, dates, group size, preferences, budget
- Output: structured itinerary JSON

2. Angular Frontend (after all backend done)

- Auth, Friends, Groups, Expense, Location map (Leaflet), AI Itinerary UI

Key Ports

- HTTPS: `https://localhost:7150`
- HTTP: `http://localhost:5283`